

SQL interview questions - Part 2

1. Question: What is the difference between INNER JOIN and LEFT JOIN?

Solution: An INNER JOIN returns only the rows where there's a match in both tables. A LEFT JOIN returns all rows from the left table and the matched rows from the right table. If there's no match, NULL values are returned for the right table's columns.

Example:

```
-- INNER JOIN
SELECT employees.name, departments.dept_name
FROM employees
INNER JOIN departments ON employees.dept_id = departments.id;

-- LEFT JOIN
SELECT employees.name, departments.dept_name
FROM employees
LEFT JOIN departments ON employees.dept_id = departments.id;
```

2. Question: Write a query to get all employees and their department names, including employees without a department.

Solution:

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.id;
```

Explanation: This LEFT JOIN will return all employees, even those without a department (their dept_name will be NULL).

3. Question: How would you find all departments that don't have any employees?

Solution:

```
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e ON d.id = e.dept_id
WHERE e.id IS NULL;
```

Explanation: This query uses a LEFT JOIN and then filters for departments where no employee ID is found, indicating no employees in that department.

4. Question: Write a query to get all employees who have a manager, along with their manager's name.

Solution:

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
INNER JOIN employees m ON e.manager_id = m.id;
```

Explanation: This is a self-join where the employees table joins with itself to match employees with their managers.

5. Question: How would you use a FULL JOIN to show all employees and all departments, regardless of whether there's a match?

Solution:

```
SELECT e.name, d.dept_name
FROM employees e
FULL JOIN departments d ON e.dept_id = d.id;
```

Explanation: FULL JOIN returns all rows from both tables, with NULL values where there's no match. Note that not all databases support FULL JOIN (e.g., MySQL doesn't).

6. Question: Write a query to get all employees and their project assignments, including employees without projects.

Solution:

```
SELECT e.name, p.project_name
FROM employees e
LEFT JOIN employee_projects ep ON e.id = ep.employee_id
LEFT JOIN projects p ON ep.project_id = p.id;
```

Explanation: This query uses multiple LEFT JOINS to ensure all employees are included, even if they're not assigned to any projects.

7. Question: How would you find all projects that have more than 3 employees assigned to them?

Solution:

```
SELECT p.project_name, COUNT(ep.employee_id) as employee_count
FROM projects p
INNER JOIN employee_projects ep ON p.id = ep.project_id
GROUP BY p.id, p.project_name
HAVING COUNT(ep.employee_id) > 3;
```

Explanation: This query uses an INNER JOIN to connect projects with employees, then groups and counts, filtering for projects with more than 3 employees.

8. Question: Write a query to get all employees and their total sales, including employees with no sales.

Solution:

```
SELECT e.name, COALESCE(SUM(s.amount), 0) as total_sales
```

```
FROM employees e
LEFT JOIN sales s ON e.id = s.employee_id
GROUP BY e.id, e.name;
```

Explanation: This LEFT JOIN ensures all employees are included. COALESCE is used to replace NULL with 0 for employees with no sales.

9. Question: How would you find the department with the highest average salary?

Solution:

```
SELECT d.dept_name, AVG(e.salary) as avg_salary
FROM departments d
INNER JOIN employees e ON d.id = e.dept_id
GROUP BY d.id, d.dept_name
ORDER BY avg_salary DESC
LIMIT 1;
```

Explanation: This query joins departments and employees, calculates the average salary per department, then orders and limits to get the highest.

10. Question: Write a query to get all employees and their direct subordinates.

Solution:

```
SELECT m.name AS manager, e.name AS subordinate
FROM employees m
LEFT JOIN employees e ON m.id = e.manager_id
ORDER BY m.name, e.name;
```

Explanation: This self-join matches employees with their direct reports. The LEFT JOIN ensures that employees with no subordinates are also included.

11. Question: How would you find all pairs of employees who work in the same department?

Solution:

```
SELECT e1.name AS employee1, e2.name AS employee2, d.dept_name
FROM employees e1
INNER JOIN employees e2 ON e1.dept_id = e2.dept_id AND e1.id < e2.id
INNER JOIN departments d ON e1.dept_id = d.id;
```

Explanation: This query uses a self-join on the employees table to pair employees in the same department. The condition `e1.id < e2.id` ensures each pair is listed only once.

12. Question: Write a query to get all departments and the name of the employee with the highest salary in each department.

Solution:

```
WITH ranked_employees AS (
    SELECT e.*, d.dept_name,
           ROW_NUMBER() OVER (PARTITION BY e.dept_id ORDER BY
e.salary DESC) as salary_rank
    FROM employees e
    INNER JOIN departments d ON e.dept_id = d.id
)
SELECT dept_name, name AS highest_paid_employee, salary
FROM ranked_employees
WHERE salary_rank = 1;
```

Explanation: This query uses a Common Table Expression (CTE) with a window function to rank employees within each department by salary, then selects the top-ranked employee for each department.

13. Question: How would you find all employees who don't have any subordinates?

Solution:

```
SELECT e.name
FROM employees e
LEFT JOIN employees sub ON e.id = sub.manager_id
WHERE sub.id IS NULL;
```

Explanation: This self-join looks for employees who don't appear as a manager for any other employee.

14. Question: Write a query to get all projects, the employees assigned to them, and the total hours worked by each employee on each project.

Solution:

```
SELECT p.project_name, e.name AS employee_name,
       COALESCE(SUM(w.hours), 0) AS total_hours
FROM projects p
LEFT JOIN employee_projects ep ON p.id = ep.project_id
LEFT JOIN employees e ON ep.employee_id = e.id
LEFT JOIN work_hours w ON ep.id = w.employee_project_id
GROUP BY p.id, p.project_name, e.id, e.name
ORDER BY p.project_name, total_hours DESC;
```

Explanation: This query uses multiple LEFT JOINS to ensure all projects are included, even if they have no employees or work hours recorded. It then calculates the total hours worked by each employee on each project.

15. Question: How would you find all employees who have a higher salary than their department's average?

Solution:

```
WITH dept_averages AS (  
    SELECT dept_id, AVG(salary) AS avg_salary  
    FROM employees  
    GROUP BY dept_id  
)  
SELECT e.name, e.salary, d.dept_name, da.avg_salary  
FROM employees e  
INNER JOIN departments d ON e.dept_id = d.id  
INNER JOIN dept_averages da ON e.dept_id = da.dept_id  
WHERE e.salary > da.avg_salary  
ORDER BY d.dept_name, e.salary DESC;
```

Explanation: This query uses a CTE to calculate the average salary for each department, then joins this with the employees and departments tables to compare each employee's salary with their department's average.